Naval Physical and Oceanographic Laboratory
DRDO, Thrikkakara, Kochi, Kerala, 682021

Internship Report

Submitted by

Meghna V S
Drishya Ajith
Rinu Sunny Mathew
Paul Kuriakose

Under the Guidance of

Dr. Sooraj K. Ambat
Scientist-F, NPOL

Internship Period: June – July 2025

Model Engineering College
Karimakkad, Thrikkakara, Edappally, Kochi, Kerala, 682021

Acknowledgements

We sincerely thank the **Naval Physical and Oceanographic Laboratory (NPOL)**, DRDO, for the opportunity to carry out our internship.
Our heartfelt gratitude goes to **Dr. Sooraj K. Ambat** for his valuable guidance and support throughout the internship.
We also thank the staff at NPOL and our faculty at **Model Engineering College**, under **APJ Abdul Kalam Technological University**, for their encouragement and assistance.

Abstract

This internship report summarizes our learning experience at the Naval Physical and Oceanographic Laboratory (NPOL) during June–July 2025. The internship focused on strengthening our understanding of fundamental machine learning concepts. We completed a Coursera course on supervised learning and discussed a widely cited academic paper, "A Tutorial on Principal Component Analysis" by Jonathon Shlens (2005). These activities gave us deep insights into linear regression, logistic regression, gradient descent, regularization, and PCA for dimensionality reduction. This report outlines the structure of our training, key technical topics covered, and the skills we acquired.

Contents

Acknowledgements **1**

Abstract **1**

1 Introduction **4**

2 About the Organization **4**

3 Machine Learning **4**
3.1 Fundamentals of Machine Learning . . . 4
3.1.1 What is Machine Learning? The Core Philosophy . . . 4
3.1.2 Core Types of Machine Learning . . . 4
3.2 Supervised Learning: Learning from Labelled Data . . . 5
3.2.1 Regression: Predicting Continuous Values . . . 5
3.2.2 Classification: Predicting Discrete Categories . . . 5
3.3 Unsupervised Learning: Discovering Hidden Patterns . . . 5
3.3.1 Clustering: Finding Natural Groupings . . . 5
3.3.2 Other Unsupervised Tasks Mentioned . . . 6
3.4 Deep Dive: Linear Regression . . . 6
3.4.1 Model Representation: The Hypothesis . . . 6
3.4.2 The Cost Function (J): Quantifying Error . . . 6
3.4.3 Gradient Descent: The Path to the Minimum . . . 7
3.5 Advanced Techniques and Practical Implementation . . . 9
3.5.1 Multiple Linear Regression & The Power of Vectorization . . . 9
3.5.2 Feature Scaling: Leveling the Playing Field . . . 10
3.5.3 Feature Engineering and Polynomial Regression . . . 10
3.6 Deep Dive: Logistic Regression for Classification . . . 11
3.6.1 The Sigmoid Function: Mapping to Probability . . . 11
3.6.2 The Decision Boundary: Making a Choice . . . 11
3.6.3 Cost Function for Logistic Regression: A Better Metric . . . 11
3.7 The Problem of Overfitting and Regularization . . . 12
3.7.1 Underfitting vs. Overfitting: The Bias-Variance Tradeoff . . . 12
3.7.2 Regularization: The Preferred Solution for Overfitting . . . 13

4 Principal Component Analysis (PCA) **13**
4.1 Introduction . . . 13
4.2 Objectives . . . 14
4.3 Motivation and Problem Statement . . . 14
4.4 Mathematical Framework . . . 14
4.5 Variance, Noise and Redundancy . . . 14
4.6 Covariance Matrix . . . 14
4.7 Solving PCA using Eigen Decomposition . . . 15
4.8 Solving PCA using SVD . . . 15
4.9 Dimensionality Reduction . . . 15
4.10 PCA on Real Datasets . . . 15

4.11 Applications of PCA 16
4.12 Limitations and Extensions 16

5 Internship Summary and Learning Outcomes **16**
5.1 Key Technical Takeaways 17
5.2 Broader Outcomes 17
5.3 Conclusion 18

A Appendix: Python Code for PCA **19**
A.1 PCA on Diabetes Dataset 19
A.2 PCA on Breast Cancer Dataset 20

1 Introduction

Our internship at NPOL was a team-based academic training program designed to enhance our knowledge in machine learning. We focused on coursework and group discussions to explore both theoretical and practical concepts. With guidance from Dr. Sooraj K. Ambat, we engaged in structured learning through online platforms and scientific literature.

2 About the Organization

The Naval Physical and Oceanographic Laboratory (NPOL) is a R&D laboratory under the Defence Research and Development Organisation (DRDO), Ministry of Defence, Government of India. NPOL specializes in cutting-edge research in sonar systems, oceanographic sensing technologies, and advanced signal processing, contributing significantly to the nation's naval defence capabilities.

3 Machine Learning

3.1 Fundamentals of Machine Learning

3.1.1 What is Machine Learning? The Core Philosophy

Machine Learning (ML) represents a paradigm shift from traditional programming.
Traditional Programming: A human programmer explicitly writes a set of rules (e.g., if-then-else logic) for the computer to follow to solve a problem. The program's logic is static.
Machine Learning: A human programmer writes an algorithm that allows the computer to learn the rules itself by processing large amounts of data. The program's logic evolves as it learns from experience.

Illustrative Example: Arthur Samuel's Checkers Program The instructor highlights this classic example to build intuition.
The Process: Instead of programming rules like "if the opponent moves here, then move there," Samuel's program played tens of thousands of games against itself.
The Learning Mechanism: It learned to associate board configurations (the data) with outcomes (win/loss). It learned that certain patterns were more likely to lead to a win (positive reinforcement) and others to a loss (negative reinforcement)
The Result: By optimizing its strategy based on this learned experience, the program surpassed its creator's ability, demonstrating the power of learning from data rather than from pre-written instructions.

3.1.2 Core Types of Machine Learning

Supervised Learning: This is like learning with a teacher or an answer key. The algorithm is given a dataset where each piece of data (X) is already tagged with the correct answer (y). Its job is to learn the general rule that connects X to y. This is the focus of the majority of the course.
Unsupervised Learning: his is like learning without a teacher. The algorithm is given a dataset of unlabeled data (X only) and must find hidden patterns, groupings, or structures on its own. It's about discovering the inherent organization within the data.

3.2 Supervised Learning: Learning from Labelled Data

Supervised learning is currently the engine behind most of the value generated by ML in industry.

3.2.1 Regression: Predicting Continuous Values

The defining characteristic of a regression problem is that the output you are trying to predict is a continuous quantity—a number that can take on any value within a given range.
Primary Example: Housing Price Prediction

- **Input (X):** A specific feature, such as the size of a house (e.g., 1,416 sq ft).
- **Output (y):** The predicted price (e.g., $232,000). The output isn't one of a few categories; it can be 232000, 232001, 198550.50, etc.

Goal: To learn a function that accurately maps the size of any house to its likely market price.

3.2.2 Classification: Predicting Discrete Categories

The goal of a classification problem is to assign an input to one of a small, finite number of predefined **categories or classes**.
Binary Classification: The simplest and most common form, where there are only two possible outcomes. The course uses the convention 0 for the "negative class" (absence of something) and 1 for the "positive class" (presence of something).
Examples:

- **Tumor Classification:** Is the tumor Malignant (1) or Benign (0)?
- **Spam Filtering:** Is the email Spam (1) or Not Spam (0)?
- **Fraud Detection:** Is the transaction Fraudulent (1) or Legitimate (0)?

Decision Making: A classification model doesn't just predict numbers; it makes a categorical decision, placing the input into a specific bucket.

3.3 Unsupervised Learning: Discovering Hidden Patterns

3.3.1 Clustering: Finding Natural Groupings

Clustering algorithms seek to partition the data into distinct groups, or **clusters**, where items within a cluster are more similar to each other than to items in other clusters.
Example: Google News In-Depth

- **Input:** Hundreds of thousands of raw, unlabeled news articles from the internet.
- **Process:** The clustering algorithm identifies articles that share common words and themes (e.g., "giant panda," "twin cubs," "zoo"). It groups them together automatically.
- **The Power:** No human told the algorithm that "panda stories" was a category. It discovered this topic cluster organically from the text itself. The topics change daily, and the algorithm adapts without human intervention.

3.3.2 Other Unsupervised Tasks Mentioned

- **Anomaly Detection:** Used to identify unusual events or outliers that don't conform to expected patterns. Crucial for fraud detection in finance.
- **Dimensionality Reduction:** A technique to compress a large dataset into a much smaller one while preserving as much important information as possible. Useful for visualization and speeding up other algorithms.

3.4 Deep Dive: Linear Regression

3.4.1 Model Representation: The Hypothesis

The model is the mathematical formula that represents the "hypothesis" about the relationship between inputs and outputs. For simple linear regression, this is a straight line.
Function: $f(x) = wx + b$
Parameters:

- **w (weight):** Represents the slope of the line. In the housing example, this is the "price per square foot." A w of 0.2 could mean the price increases by $200 for every additional square foot.
- **b (bias):** Represents the y-intercept. In the housing example, this is the "base price" of a house, even if it had zero size.

The Learning Goal: To find the specific values of w and b that produce a line that best fits the training data points.

Figure 1: Graphical Representation of the Hypothesis in a Linear Regression Model

3.4.2 The Cost Function (J): Quantifying Error

How do we define the "best" line? We define it as the line that minimizes the total error. The cost function, $J(w, b)$, is the tool for measuring this error.

Mean Squared Error (MSE):

$$J(w,b) = \frac{1}{2m}\sum_{i=1}^{m}((wx_i + b) - y_i)^2$$

- $(wx_i + b)$ is the model's prediction for the i-th example.
- y_i is the actual, true value for that example.
- (prediction − actual) is the error for that one example.
- The error is squared. This makes all errors positive and penalizes larger errors more heavily.
- The total cost is averaged across all m training examples.
- The factor $\frac{1}{2}$ is used for simplifying the derivative computation later.

Visualization: The cost function J can be pictured as a 3D bowl. The horizontal axes represent the possible values of w and b, and the vertical axis represents the cost. The lowest point of this bowl is the global minimum—the pair of (w, b) that gives the lowest possible error.

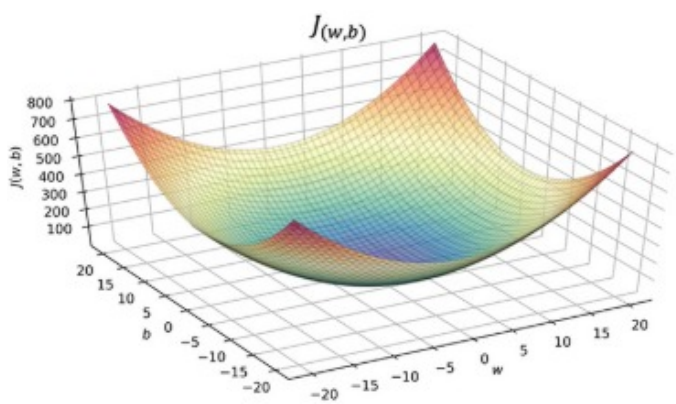

Figure 2: Visualization of the Cost Function Landscape: Mean Squared Error (MSE) as a Function of Model Parameters

3.4.3 Gradient Descent: The Path to the Minimum

Gradient Descent is the iterative optimization algorithm that "walks" down the slope of the cost function to find the lowest point.

The Analogy: You are standing on the surface of the cost function bowl, blindfolded.

- You feel the slope of the ground beneath your feet. The derivative tells you the direction of steepest descent.

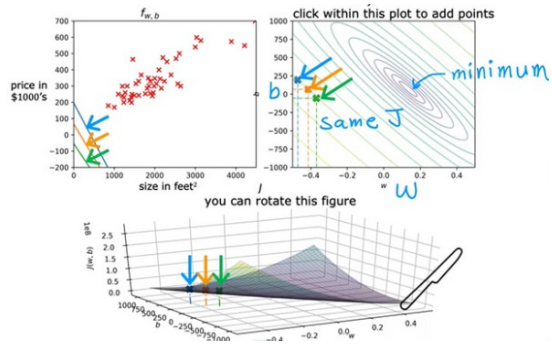

Figure 3: Gradient Descent Process: Iterative Minimization of the Cost Function by Moving Towards the Global Minimum

- You take a step in that downhill direction. The size of your step is controlled by the learning rate α.
- You repeat this process. As you get closer to the bottom, the slope becomes less steep, so your steps automatically become smaller, allowing you to converge smoothly to the minimum.

The Update Rule: On each iteration, you simultaneously update w and b:

$$w = w - \alpha \cdot \frac{\partial J}{\partial w}$$

$$b = b - \alpha \cdot \frac{\partial J}{\partial b}$$

The Derivative Term ($\frac{\partial J}{\partial w}$): This is the crucial part that calculates the slope. Its value is proportional to the error, so large errors lead to large updates, and small errors lead to small updates.

The Learning Rate (α): A hyperparameter you must choose. Too small, and convergence takes forever. Too large, and you risk "overshooting" the minimum and diverging. A typical debugging process involves trying different values (e.g., 0.001, 0.01, 0.1) and plotting the cost function over iterations to see which one works best.

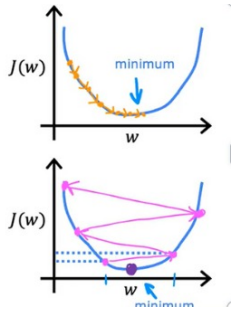

Figure 4: Effect of Learning Rate on Gradient Descent: Top—Learning Rate Too Small, Leading to Slow Convergence; Bottom—Learning Rate Too Large, Causing Divergence

3.5 Advanced Techniques and Practical Implementation

3.5.1 Multiple Linear Regression & The Power of Vectorization

The Model: When we have multiple input features—such as size x_1, number of bedrooms x_2, and house age x_3—our model becomes:

$$f(x) = w_1x_1 + w_2x_2 + w_3x_3 + b$$

Vectorized Notation:

$$f(x) = \mathbf{w} \cdot \mathbf{x} + b$$

where

$$\mathbf{w} = [w_1, w_2, w_3], \quad \mathbf{x} = [x_1, x_2, x_3]$$

Vectorization in Practice: Instead of writing a slow for loop in code, we use a numerical library like NumPy. The command `np.dot(w, x) + b` performs the entire calculation at once. This is not just syntactic sugar; it leverages low-level, parallelized hardware instructions (SIMD) to execute computations orders of magnitude faster than a Python loop, which is essential for working with large datasets.

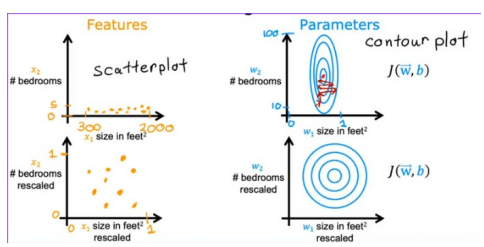

Figure 5: Illustration of Multiple Features and Vectorized Computation in Linear Regression

3.5.2 Feature Scaling: Leveling the Playing Field

The Problem: The Problem: If feature x_1 (size) ranges from 300–2000 and x_2 (bedrooms) ranges from 0–5, then the cost function becomes a highly skewed, narrow canyon. Gradient descent struggles to navigate it.

The Solution: Rescale all features to a similar, smaller range (e.g., approximately -1 to 1). This reshapes the cost function "canyon" into a more symmetrical "bowl," allowing gradient descent to find a much more direct path to the minimum. As a result, convergence is dramatically accelerated.

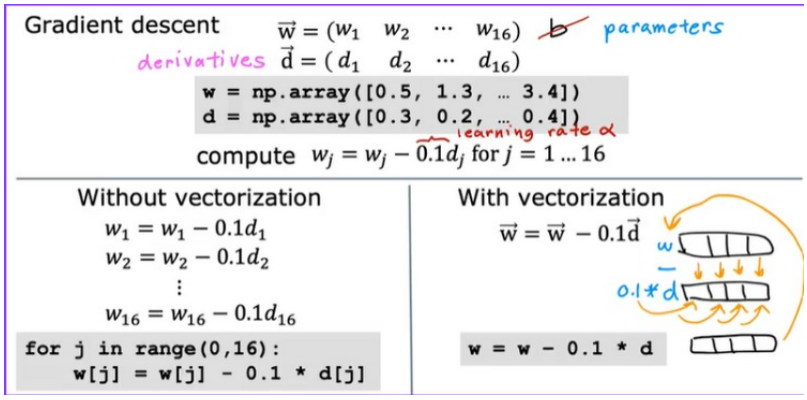

Figure 6: Impact of Feature Scaling on the Shape of the Cost Function Landscape in Linear Regression

- **Mean Normalization:** $x' = \frac{x - \text{mean}(x)}{\max(x) - \min(x)}$
- **Z-score Normalization:** $x' = \frac{x - \mu}{\sigma}$

3.5.3 Feature Engineering and Polynomial Regression

The Idea: Linear regression can still model non-linear relationships by adding new polynomial features.
Example: Instead of using only x, we add x^2, x^3, etc.:

$$f(x) = w_1 x + w_2 x^2 + w_3 x^3 + \cdots + b$$

Why It Works:

- The model remains linear in terms of its parameters.
- It enables capturing curved patterns in data without switching to a different learning algorithm.

Important Consideration: As higher-degree terms grow rapidly, feature scaling becomes even more crucial to maintain numerical stability and fast convergence.

3.6 Deep Dive: Logistic Regression for Classification

3.6.1 The Sigmoid Function: Mapping to Probability

The core of logistic regression is the sigmoid function, which solves linear regression's problem of producing outputs outside the (0, 1) range.
Function: $g(z) = \frac{1}{1+e^{-z}}$
Mechanism: It takes the linear output $z = \mathbf{w} \cdot \mathbf{x} + b$ and "squashes" it into a value between 0 and 1.
Interpretation: This squashed value $f(x) = g(z)$ is now interpreted as the probability that the output class is 1. $P(y = 1|x)$.

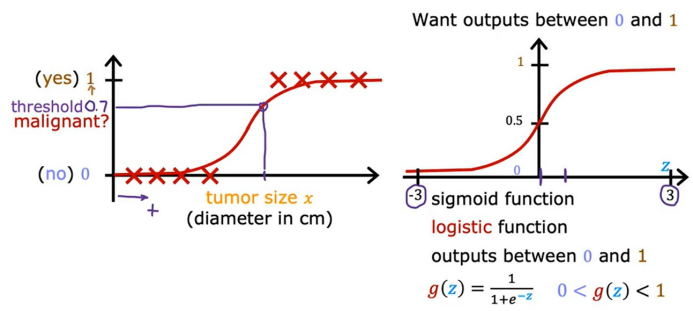

Figure 7: Graphical Representation of the Sigmoid Function, Used in Logistic Regression for Mapping Inputs to Probabilities

3.6.2 The Decision Boundary: Making a Choice

The decision boundary is the conceptual line or surface that separates the classes.

Logic: The model predicts $y = 1$ if it estimates the probability is ≥ 0.5.

Connection to z: Looking at the sigmoid curve, $g(z) \geq 0.5$ occurs precisely when $z \geq 0$.

The Boundary Equation: Therefore, the decision boundary is the line where $z = 0$, or $\mathbf{w} \cdot \mathbf{x} + b = 0$. The algorithm's job is to find the parameters $\mathbf{w}$ and b that place this boundary in the best possible location to separate the data.

3.6.3 Cost Function for Logistic Regression: A Better Metric

MSE is not suitable here because it creates a non-convex cost function with many local minima. A new, convex cost function is needed.

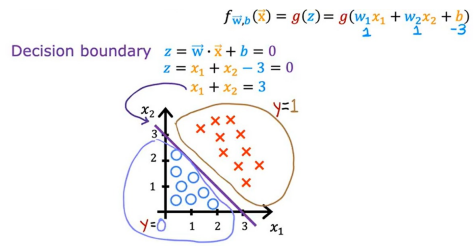

Figure 8: Linear Decision Boundary: Separating Data Points Using a Straight Line in Binary Classification

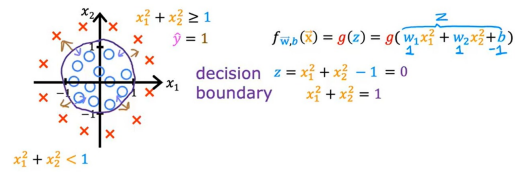

Figure 9: Non-Linear Decision Boundary: Separating Data Points Using a Curved Boundary in Binary Classification

s

Logistic Loss (or Log Loss):

$$\text{Loss} = -y \cdot \log(f(x)) - (1 - y) \cdot \log(1 - f(x))$$

Intuition: This function penalizes confident mistakes extremely harshly. If the true label $y = 1$ but the model predicts a probability near 0 ($f(x) \to 0$), the term $-\log(f(x))$ goes to infinity. This powerful penalty forces the model to learn correctly.

3.7 The Problem of Overfitting and Regularization

3.7.1 Underfitting vs. Overfitting: The Bias-Variance Tradeoff

This is a fundamental challenge in machine learning.

Underfitting (High Bias): The model is too simple (e.g., a linear model for complex data). It has a strong, preconceived "bias" that the data is simple, and it cannot even fit the training data well.

Overfitting (High Variance): The model is too complex (e.g., a very high-degree polynomial). It fits the training data perfectly, memorizing its noise and idiosyncrasies. Its predictions are highly "variable" and change drastically with small changes in the training data. It fails to generalize to new data.

The Goal: A "just right" model that is flexible enough to capture the true underlying pattern without memorizing the noise.

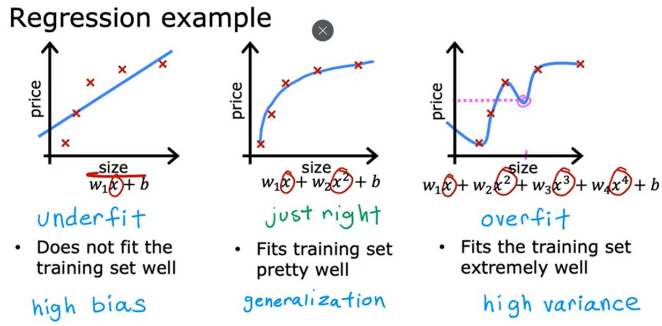

Figure 10: Illustration of Underfitting and Overfitting: Balancing Model Complexity for Optimal Generalization in Machine Learning

3.7.2 Regularization: The Preferred Solution for Overfitting

Regularization is a technique that helps prevent overfitting by discouraging the model from becoming too complex.

The Mechanism: It adds a penalty term to the cost function that increases with the size of the model parameters (w).

Modified Cost:

$$J(w, b) = (\text{Original Cost}) + \frac{\lambda}{2m} \sum_j w_j^2$$

The λ (Lambda) Parameter: This is the regularization parameter, a "knob" that controls the strength of the penalty.

- If λ is large, the penalty is high, forcing the weights w to be very small. This results in a simpler model that may underfit.
- If $\lambda = 0$, there is no regularization.

The Intuition: Regularization creates a trade-off. The model must now minimize both the error on the training set and the size of its parameters. This forces it to learn simpler, smoother functions that tend to generalize better. The gradient descent update for w effectively shrinks the value of w slightly on every iteration, leading to the desired effect. This powerful technique is used for both linear and logistic regression.

4 Principal Component Analysis (PCA)

4.1 Introduction

PCA is a cornerstone of modern data analysis and linear algebra. It is used in domains like neuroscience, meteorology, and machine learning to find patterns, compress data, and denoise signals. This report follows the approach of Jonathon Shlens' tutorial, explaining PCA from the ground up.

4.2 Objectives

- Develop an intuitive and mathematical understanding of PCA.
- Explore linear algebra tools like eigen decomposition and SVD.
- Understand when PCA works and where it can fail.

4.3 Motivation and Problem Statement

In complex data, true dynamics can be obscured by noise and redundancy. For example, measuring the position of a mass on a spring with three off-angle cameras results in a redundant dataset. PCA helps find the most relevant direction (principal component) revealing the true one-dimensional motion.

4.4 Mathematical Framework

Given a data matrix $X \in \mathbb{R}^{m \times n}$, PCA finds a transformation matrix P such that:

$$PX = Y$$

Where:

- X: Original data matrix.
- P: Linear transformation matrix (rotation and scaling).
- Y: Transformed data in the new basis.

Each column of Y represents projections of the data onto new basis vectors.

4.5 Variance, Noise and Redundancy

PCA relies on:

- Variance as a proxy for signal.
- Redundancy captured through covariance.
- Noise being uncorrelated, low-variance structure.

4.6 Covariance Matrix

$$C_X = \frac{1}{n-1} X X^T$$

- Diagonal terms represent feature variances.
- Off-diagonal terms represent covariances.

PCA aims to find a new basis P such that:

$$C_Y = P C_X P^T \quad \text{is diagonal}$$

4.7 Solving PCA using Eigen Decomposition

Let $C_X = \frac{1}{n-1} X X^T$. Perform eigen decomposition:

$$C_X = EDE^T$$

Where:

- E: Matrix of eigenvectors (principal components).
- D: Diagonal matrix of eigenvalues (variances).

Then:

$$Y = E^T X$$

4.8 Solving PCA using SVD

Singular Value Decomposition (SVD) offers an alternative:

$$X = U \Sigma V^T$$

Where:

- U: Orthonormal left singular vectors.
- Σ: Diagonal matrix of singular values.
- V: Orthonormal right singular vectors (principal components).

We define:

$$Y = \frac{1}{\sqrt{n-1}} X^T \quad \Rightarrow \quad C_X = Y^T Y$$

Then V contains the principal components.

4.9 Dimensionality Reduction

After computing PCA, select the top k principal components with the largest eigenvalues:

- Reduces dimensionality.
- Retains maximum variance.
- Filters out noise and redundancy.

4.10 PCA on Real Datasets

To reinforce the theoretical understanding, PCA was applied to two real-world datasets: the Breast Cancer dataset and the Diabetes dataset from scikit-learn. Each dataset was normalized and reduced using PCA. The eigenvalues, cumulative explained variance, and projections of the data onto the principal components were visualized.

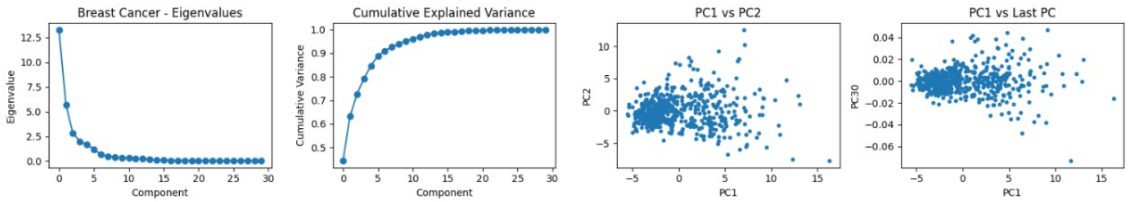

Figure 11: PCA on Breast Cancer Dataset: Eigenvalues, Cumulative Variance, PC1 vs PC2, PC1 vs Last PC

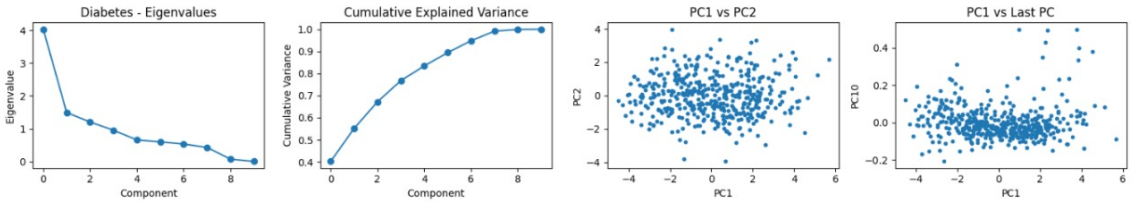

Figure 12: PCA on Diabetes Dataset: Eigenvalues, Cumulative Variance, PC1 vs PC2, PC1 vs Last PC

4.11 Applications of PCA

- Data visualization and compression
- Signal and image denoising
- Feature extraction in machine learning
- Face recognition using eigenfaces

4.12 Limitations and Extensions

- PCA assumes linearity and Gaussian-distributed data.
- PCA may fail on exponential or non-linear data.
- Extensions include Kernel PCA and Independent Component Analysis (ICA).

5 Internship Summary and Learning Outcomes

Our four-week internship at the Naval Physical and Oceanographic Laboratory (NPOL) proved to be far more than an academic requirement—it became a structured launch-pad into the real-world practice of modern machine learning and a gateway into the world of defence research. Under the mentorship of Dr. Sooraj K. Ambat, we engaged in rigorous self-study, peer learning, and immersive discussion sessions that took us from theoretical foundations to practical implementation, and ultimately, to complete data analysis pipelines.

5.1 Key Technical Takeaways

Supervised Learning Workflow We built regression and classification models from scratch, implemented them using vectorized `NumPy` code, and optimized them using gradient descent. This hands-on approach taught us not just the "how," but also the "why" behind model behavior. We also explored critical aspects such as feature scaling, regularization, and the bias–variance tradeoff, which are essential for building reliable, generalizable ML systems.

Unsupervised Learning Intuition We dove into clustering, anomaly detection, and dimensionality reduction—experiencing firsthand how machine learning can uncover hidden patterns without explicit supervision.

End-to-End PCA Pipeline Our exploration of Principal Component Analysis (PCA) was both mathematical and visual. Implementing PCA via both eigen-decomposition and SVD improved our understanding of linear algebra and statistical thinking. By applying PCA to real datasets like Breast Cancer and Diabetes, we witnessed the power of dimensionality reduction in filtering noise and extracting signal.

5.2 Broader Outcomes

Workplace Immersion in a Defence Research Environment One of the most unique and eye-opening aspects of the internship was the opportunity to work inside a high-security, government defence organization. We observed firsthand the work culture, professionalism, and technological applications that directly serve India's naval defence capabilities. From sophisticated sonar systems to oceanographic sensing platforms, we gained valuable exposure to practical tools and real-world systems used by the Indian Navy.

Expanding Our Horizons Beyond Machine Learning Our structured discussion sessions under Dr. Sooraj were transformative. They didn't just cover machine learning—they opened us up to new domains of technology, emerging research topics, and ways of thinking that go far beyond textbooks. Whether it was understanding scientific reasoning, exploring broader applications of AI, or reflecting on the role of engineering in society, every session left us intellectually richer.

Research Readiness & Tool-Chain Proficiency We are now equipped to translate abstract ML theory into reproducible experiments. Exposure to modern tools like `Python`, `NumPy`, and `scikit-learn` gave us hands-on experience with industry-standard workflows, data preprocessing, vectorization, and visualization.

Team Science Mindset Collaborating as a team, reviewing literature, and performing peer code reviews helped develop a culture of scientific collaboration—essential for large-scale, interdisciplinary research projects, particularly in defence R&D environments.

Connections and Mentorship We were fortunate to make valuable professional connections, engage with experienced researchers, and gain meaningful insights into both academic and applied research. The mentorship we received from Dr. Sooraj K. Ambat left a lasting impact—not only

by guiding our technical learning, but by instilling in us a deep curiosity, a structured approach to problem-solving, and the mindset of a researcher.

5.3 Conclusion

In summary, this internship has been a transformative journey—technically rigorous, intellectually expansive, and personally inspiring. It has transformed our understanding of machine learning from a set of isolated ideas into a deployable, adaptable toolkit grounded in real-world applicability. More importantly, it introduced us to the research culture, work environment, and national significance of defence technology.

We leave this experience more confident, more capable, and more committed to contributing to India's scientific and technological progress.

A Appendix: Python Code for PCA

A.1 PCA on Diabetes Dataset

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
from sklearn.datasets import load_diabetes

# Load and normalize Diabetes dataset
data = load_diabetes().data
data = StandardScaler().fit_transform(data)

# Apply PCA
pca = PCA(n_components=data.shape[1])
pca_result = pca.fit_transform(data)

# Eigenvalues and cumulative variance
eigenvalues = pca.explained_variance_
cum_var = np.cumsum(pca.explained_variance_ratio_)

# Plotting
fig, axs = plt.subplots(1, 4, figsize=(16, 3))

# Plot eigenvalues
axs[0].plot(eigenvalues, marker='o')
axs[0].set_title('Diabetes␣-␣Eigenvalues')
axs[0].set_xlabel('Component')
axs[0].set_ylabel('Eigenvalue')

# Plot cumulative variance
axs[1].plot(cum_var, marker='o')
axs[1].set_title('Cumulative␣Explained␣Variance')
axs[1].set_xlabel('Component')
axs[1].set_ylabel('Cumulative␣Variance')

# PC1 vs PC2
axs[2].scatter(pca_result[:, 0], pca_result[:, 1], s=10)
axs[2].set_title('PC1␣vs␣PC2')
axs[2].set_xlabel('PC1')
axs[2].set_ylabel('PC2')

# PC1 vs last PC
axs[3].scatter(pca_result[:, 0], pca_result[:, -1], s=10)
axs[3].set_title('PC1␣vs␣Last␣PC')
axs[3].set_xlabel('PC1')
axs[3].set_ylabel(f'PC{data.shape[1]}')

plt.tight_layout()
```

```
plt.show()
```

A.2 PCA on Breast Cancer Dataset

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
from sklearn.datasets import load_breast_cancer

# Load and normalize Breast Cancer dataset
data = load_breast_cancer().data
data = StandardScaler().fit_transform(data)

# Apply PCA
pca = PCA(n_components=data.shape[1])
pca_result = pca.fit_transform(data)

# Eigenvalues and cumulative variance
eigenvalues = pca.explained_variance_
cum_var = np.cumsum(pca.explained_variance_ratio_)

# Plotting
fig, axs = plt.subplots(1, 4, figsize=(16, 3))

# Plot eigenvalues
axs[0].plot(eigenvalues, marker='o')
axs[0].set_title('Breast␣Cancer␣-␣Eigenvalues')
axs[0].set_xlabel('Component')
axs[0].set_ylabel('Eigenvalue')

# Plot cumulative variance
axs[1].plot(cum_var, marker='o')
axs[1].set_title('Cumulative␣Explained␣Variance')
axs[1].set_xlabel('Component')
axs[1].set_ylabel('Cumulative␣Variance')

# PC1 vs PC2
axs[2].scatter(pca_result[:, 0], pca_result[:, 1], s=10)
axs[2].set_title('PC1␣vs␣PC2')
axs[2].set_xlabel('PC1')
axs[2].set_ylabel('PC2')

# PC1 vs last PC
axs[3].scatter(pca_result[:, 0], pca_result[:, -1], s=10)
axs[3].set_title('PC1␣vs␣Last␣PC')
axs[3].set_xlabel('PC1')
axs[3].set_ylabel(f'PC{data.shape[1]}')
```

```
plt.tight_layout()
plt.show()
```